

LAB ASSIGNMENT-11.3

2303A51337

BATCH-10

TASK-1: Smart Contact Manager (Arrays & Linked Lists)

PROMPT: (using array)

Python program for a Smart Contact Manager. Implement using Array (Python list). Each contact should have: Name, Phone Number. Implement operations: Add contact, Search contact, Delete contact, Display contacts.

CODE:

```
class ContactManagerArray:

    def __init__(self):

        self.contacts = []

    def add_contact(self, name, phone):

        self.contacts.append({"name": name, "phone": phone})

    def search_contact(self, name):

        for contact in self.contacts:

            if contact["name"] == name:

                return contact["phone"]

        return "Contact not found"

    def delete_contact(self, name):

        for i, contact in enumerate(self.contacts):

            if contact["name"] == name:

                del self.contacts[i]

                return "Contact deleted"

        return "Contact not found"

    def display_contacts(self):

        return [{"contact['name']}: {contact['phone']}" for contact in self.contacts]}

# Testing the Array implementation

contact_manager_array = ContactManagerArray()

contact_manager_array.add_contact("ammu", "555-555-5555")
```

```

contact_manager_array.add_contact("nithu", "444-444-4444")

print(contact_manager_array.display_contacts())

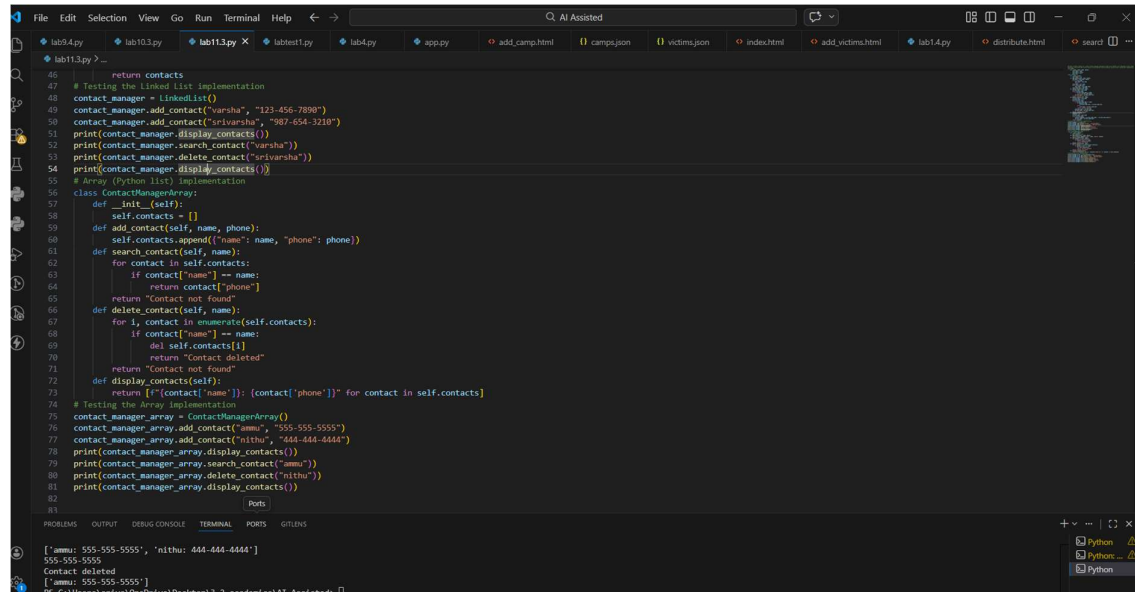
print(contact_manager_array.search_contact("ammu"))

print(contact_manager_array.delete_contact("nithu"))

print(contact_manager_array.display_contacts())

```

OUTPUT:



The screenshot shows a code editor with a Python script. The script defines two classes: `LinkedList` and `ContactManagerArray`. The `LinkedList` class has methods for adding, searching, deleting, and displaying contacts. The `ContactManagerArray` class is a wrapper around the `LinkedList` class. The script tests both implementations. The terminal output shows the results of these tests.

```

46     return contacts
47 # Testing the linked list implementation
48 contact_manager = LinkedList()
49 contact_manager.add_contact("varsha", "122-456-7890")
50 contact_manager.add_contact("sivashva", "987-654-3210")
51 print(contact_manager.display_contacts())
52 print(contact_manager.search_contact("varsha"))
53 print(contact_manager.delete_contact("sivashva"))
54 print(contact_manager.display_contacts())
55 # Array (Python list) implementation
56 class ContactManagerArray:
57     def __init__(self):
58         self.contacts = []
59     def add_contact(self, name, phone):
60         self.contacts.append({"name": name, "phone": phone})
61     def search_contact(self, name):
62         for contact in self.contacts:
63             if contact["name"] == name:
64                 return contact["phone"]
65         return "Contact not found"
66     def delete_contact(self, name):
67         for i, contact in enumerate(self.contacts):
68             if contact["name"] == name:
69                 del self.contacts[i]
70         return "Contact deleted"
71     def display_contacts(self):
72         return [{"name": name, "phone": phone} for name, phone in self.contacts]
73 # Testing the Array implementation
74 contact_manager_array = ContactManagerArray()
75 contact_manager_array.add_contact("ammu", "555-555-5555")
76 contact_manager_array.add_contact("nithu", "444-444-4444")
77 print(contact_manager_array.display_contacts())
78 print(contact_manager_array.search_contact("ammu"))
79 print(contact_manager_array.delete_contact("nithu"))
80 print(contact_manager_array.display_contacts())
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

Terminal Output:

```

["ammu: 555-555-5555", "nithu: 444-444-4444"]
555-555-5555
Contact deleted
["ammu: 555-555-5555"]

```

PROMPT:(Using linked list)

Python program for a Smart Contact Manager. Implement using Linked List (Node and LinkedList classes). Each contact should have: Name, Phone Number. Implement operations: Add contact, Search contact, Delete contact, Display contacts.

Code:

class Node:

```

def __init__(self, name, phone):

    self.name = name

    self.phone = phone

    self.next = None

```

class LinkedList:

```

def __init__(self):

    self.head = None

def add_contact(self, name, phone):

```

```
new_node = Node(name, phone)

if not self.head:

    self.head = new_node

    return

last_node = self.head

while last_node.next:

    last_node = last_node.next

last_node.next = new_node

def search_contact(self, name):

    current_node = self.head

    while current_node:

        if current_node.name == name:

            return current_node.phone

        current_node = current_node.next

    return "Contact not found"

def delete_contact(self, name):

    current_node = self.head

    previous_node = None

    while current_node:

        if current_node.name == name:

            if previous_node:

                previous_node.next = current_node.next

            else:

                self.head = current_node.next

            return "Contact deleted"

        previous_node = current_node

        current_node = current_node.next

    return "Contact not found"

def display_contacts(self):

    contacts = []
```

```

current_node = self.head

while current_node:

    contacts.append(f"{current_node.name}: {current_node.phone}")

    current_node = current_node.next

return contacts

contact_manager = LinkedList()

contact_manager.add_contact("varsha", "123-456-7890")

contact_manager.add_contact("srivarsha", "987-654-3210")

print(contact_manager.display_contacts())

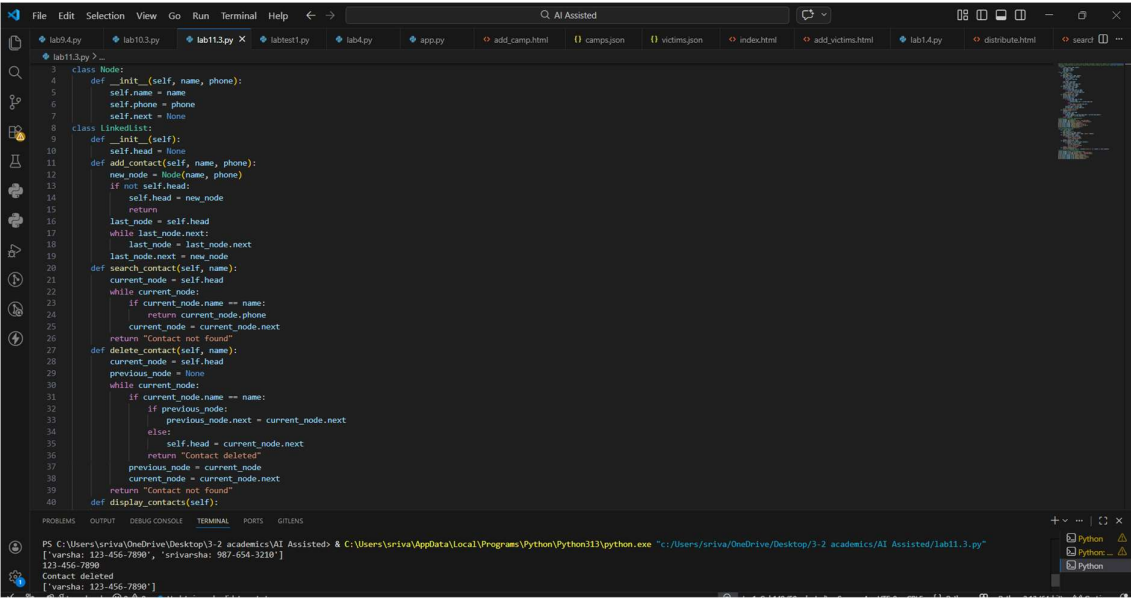
print(contact_manager.search_contact("varsha"))

print(contact_manager.delete_contact("srivarsha"))

print(contact_manager.display_contacts())

```

Output:



The screenshot shows a VS Code editor with a Python file named `lab11.3.py`. The code defines a `LinkedList` class with methods for adding, searching, deleting, and displaying contacts. The terminal output shows the results of running the script:

```

["varsha: 123-456-7890", "srivarsha: 987-654-3210"]
Contact deleted
["varsha: 123-456-7890"]

```

Analysis:

- Array allows faster insertion using `append()` $O(1)$, while Linked List insertion at the end requires traversal $O(n)$.
- In an Array, deleting a contact requires shifting elements $O(n)$. In a Linked List, deletion is done by changing pointers but still requires traversal $O(n)$.
- Arrays use less memory but have fixed structure, whereas Linked Lists use extra memory for pointers but provide dynamic memory allocation and better flexibility.

TASK-2

PROMPT:

Create a Python program for a Library Book Search System. Implement a Queue (FIFO) to manage book borrow requests. Each request should include: Name, Book Title, User Type (Student or Faculty). Implement queue operations: enqueue, dequeue, display.

CODE:

```
class BookRequest:

    def __init__(self, name, book_title, user_type):

        self.name = name

        self.book_title = book_title

        self.user_type = user_type

class LibraryQueue:

    def __init__(self):

        self.queue = []

    def enqueue(self, request):

        self.queue.append(request)

    def dequeue(self):

        if not self.queue:

            return "No requests in the queue"

        for i, request in enumerate(self.queue):

            if request.user_type == "Faculty":

                return self.queue.pop(i)

    def display(self):

        return [f"{request.name} requested '{request.book_title}' ({request.user_type})" for request
in self.queue]

# Testing the Library Queue with Priority

library_queue = LibraryQueue()

library_queue.enqueue(BookRequest("Alice", "Python Programming", "Student"))

library_queue.enqueue(BookRequest("Bob", "Data Structures", "Faculty"))

library_queue.enqueue(BookRequest("Charlie", "Algorithms", "Student"))
```

```

print("Current Queue:")

print(library_queue.display())

print("\nProcessing Requests:")

print(library_queue.dequeue().name) # Bob (Faculty)

print(library_queue.dequeue().name) # Alice (Student)

print(library_queue.dequeue().name) # Charlie (Student)

print("\nCurrent Queue after processing:")

print(library_queue.display())

```

OUTPUT:

```

class BookRequest:
    def __init__(self, name, book_title, user_type):
        self.name = name
        self.book_title = book_title
        self.user_type = user_type

class LibraryQueue:
    def __init__(self):
        self.queue = []

    def enqueue(self, request):
        self.queue.append(request)

    def dequeue(self):
        if not self.queue:
            return "No requests in the queue"

        for i, request in enumerate(self.queue):
            if request.user_type == "Faculty":
                return self.queue.pop(i)

    def display(self):
        return [{"request.name": request.name, "request.book_title": request.book_title, "request.user_type": request.user_type} for request in self.queue]

# Testing the Library Queue with Priority
library_queue = LibraryQueue()
library_queue.enqueue(BookRequest("Alice", "Python Programming", "Student"))
library_queue.enqueue(BookRequest("Bob", "Data Structures", "Faculty"))
library_queue.enqueue(BookRequest("Charlie", "Algorithms", "Student"))

print("Current Queue:")
print(library_queue.display())

print("\nProcessing Requests:")
print(library_queue.dequeue().name) # Bob (Faculty)
print(library_queue.dequeue().name) # Alice (Student)
print(library_queue.dequeue().name) # Charlie (Student)

print("\nCurrent Queue after processing:")
print(library_queue.display())

```

PROBLEMS **OUTPUT** **DEBUG CONSOLE** **TERMINAL** **PORTS** **GIT LENS**

```

No requests in the queue
PS C:\Users\sriva\OneDrive\Desktop\3-2 academics\AI Assisted> C:\Users\sriva\AppData\Local\Programs\Python\Python313\python.exe "C:\Users\sriva\OneDrive\Desktop\3-2 academics\AI Assisted\lab1.3.py"
Current Queue:
["Alice requested 'Python Programming' (Student)", "Bob requested 'Data Structures' (Faculty)", "Charlie requested 'Algorithms' (Student)"]

Processing Requests:
Bob
Alice
Charlie

Current Queue after processing:
[]

```

- Analysis:**

The system stores book requests in a queue using a Python list. Requests are added using enqueue and processed using dequeue.
- The dequeue method checks the queue and processes Faculty requests first, giving them higher priority over Student requests.
- When tested with mixed requests (Student and Faculty), the system correctly processes Faculty requests before Student requests, ensuring proper prioritization.

TASK-3:**PROMPT:**

Create a Python program for an Emergency Help Desk Ticket System using a Stack (LIFO). Implement a Stack to manage support tickets. Each ticket should contain: Ticket ID, Issue description, User name.

Implement stack operations: push(ticket), pop(), peek(), is_empty(), is_full() (if stack size is limited) Simulate at least five tickets being raised and resolved to demonstrate LIFO behavior.

CODE:

```
class Ticket:
```

```
    def __init__(self, ticket_id, issue_description, user_name):
```

```
        self.ticket_id = ticket_id
```

```
        self.issue_description = issue_description
```

```
        self.user_name = user_name
```

```
class HelpDeskStack:
```

```
    def __init__(self, max_size=10):
```

```
        self.stack = []
```

```
        self.max_size = max_size
```

```
    def push(self, ticket):
```

```
        if len(self.stack) >= self.max_size:
```

```
            return "Stack is full. Cannot add more tickets."
```

```
        self.stack.append(ticket)
```

```
    def pop(self):
```

```
        if self.is_empty():
```

```
            return "Stack is empty. No tickets to resolve."
```

```
        return self.stack.pop()
```

```
    def peek(self):
```

```
        if self.is_empty():
```

```
            return "Stack is empty. No tickets to peek."
```

```
        return self.stack[-1]
```

```
    def is_empty(self):
```

```
        return len(self.stack) == 0
```

```

def is_full(self):

    return len(self.stack) >= self.max_size

# Simulating ticket creation and resolution

help_desk = HelpDeskStack(max_size=5)

help_desk.push(Ticket(1, "Cannot access email", "Alice"))

help_desk.push(Ticket(2, "Computer won't start", "Bob"))

help_desk.push(Ticket(3, "Software installation issue", "Charlie"))

help_desk.push(Ticket(4, "Network connectivity problem", "David"))

help_desk.push(Ticket(5, "Printer not working", "Eve"))

print("Current Tickets in Stack:")

for ticket in help_desk.stack:

    print(f"Ticket ID: {ticket.ticket_id}, Issue: {ticket.issue_description}, User: {ticket.user_name}")

print("\nResolving Tickets:")

while not help_desk.is_empty():

    resolved_ticket = help_desk.pop()

    print(f"Resolved Ticket ID: {resolved_ticket.ticket_id}, Issue:
{resolved_ticket.issue_description}, User: {resolved_ticket.user_name}")

```

OUTPUT:

```

class HelpDeskStack:
    def __init__(self, max_size):
        self.stack = []
        self.max_size = max_size

    def pop(self):
        if self.is_empty():
            return "Stack is empty. No tickets to resolve."
        return self.stack.pop()

    def peek(self):
        if self.is_empty():
            return "Stack is empty. No tickets to peek."
        return self.stack[-1]

    def is_empty(self):
        return len(self.stack) == 0

    def is_full(self):
        return len(self.stack) >= self.max_size

# Simulating ticket creation and resolution
help_desk = HelpDeskStack(max_size=5)
help_desk.push(Ticket(1, "Cannot access email", "Alice"))
help_desk.push(Ticket(2, "Computer won't start", "Bob"))
help_desk.push(Ticket(3, "Software installation issue", "Charlie"))
help_desk.push(Ticket(4, "Network connectivity problem", "David"))
help_desk.push(Ticket(5, "Printer not working", "Eve"))

print("Current Tickets in Stack:")

for ticket in help_desk.stack:
    print(f"Ticket ID: {ticket.ticket_id}, Issue: {ticket.issue_description}, User: {ticket.user_name}")

print("\nResolving Tickets:")

while not help_desk.is_empty():
    resolved_ticket = help_desk.pop()
    print(f"Resolved Ticket ID: {resolved_ticket.ticket_id}, Issue: {resolved_ticket.issue_description}, User: {resolved_ticket.user_name}")

```

PS C:\Users\sriva\OneDrive\Desktop\3-2 academics\AI Assisted> & C:\Users\sriva\AppData\Local\Programs\Python\Python311\python.exe "c:\Users\sriva\OneDrive\Desktop\3-2 academics\AI Assisted\lab11.3.py"

```

Current Tickets in Stack:
Ticket ID: 1, Issue: Cannot access email, User: Alice
Ticket ID: 2, Issue: Computer won't start, User: Bob
Ticket ID: 3, Issue: Software installation issue, User: Charlie
Ticket ID: 4, Issue: Network connectivity problem, User: David
Ticket ID: 5, Issue: Printer not working, User: Eve

Resolving Tickets:
Resolved Ticket ID: 5, Issue: Printer not working, User: Eve
Resolved Ticket ID: 4, Issue: Network connectivity problem, User: David
Resolved Ticket ID: 3, Issue: Software installation issue, User: Charlie
Resolved Ticket ID: 2, Issue: Computer won't start, User: Bob
Resolved Ticket ID: 1, Issue: Cannot access email, User: Alice
PS C:\Users\sriva\OneDrive\Desktop\3-2 academics\AI Assisted>

```


Analysis:

- The stack processes support tickets using the Last-In, First-Out (LIFO) principle, meaning the most recently raised ticket is resolved first.
- The system uses basic stack operations such as push() to add tickets, pop() to resolve tickets, peek() to view the top ticket, and is_empty() to check if the stack has no tickets.

TASK-4:**PROMPT:**

Create a Python program for an Emergency Help Desk using a Stack (LIFO).Implement stack.

Each ticket should contain ticket ID, user name, and issue description.Simulate at least five tickets being raised and resolved .

CODE:

```
class Ticket:
```

```
    def __init__(self, ticket_id, user_name, issue_description):  
  
        self.ticket_id = ticket_id  
  
        self.user_name = user_name  
  
        self.issue_description = issue_description
```

```
class HelpDeskStack:
```

```
    def __init__(self, max_size=10):  
  
        self.stack = []  
  
        self.max_size = max_size  
  
    def push(self, ticket):  
  
        if len(self.stack) >= self.max_size:  
  
            return "Stack is full. Cannot add more tickets."  
  
        self.stack.append(ticket)  
  
    def pop(self):  
  
        if self.is_empty():  
  
            return "Stack is empty. No tickets to resolve."  
  
        return self.stack.pop()  
  
    def peek(self):  
  
        if self.is_empty():  
  
            return "Stack is empty. No tickets to peek."
```

```

        return self.stack[-1]

def is_empty(self):

    return len(self.stack) == 0

def is_full(self):

    return len(self.stack) >= self.max_size

help_desk = HelpDeskStack(max_size=5)

help_desk.push(Ticket(1, "Alice", "Cannot access email"))

help_desk.push(Ticket(2, "Bob", "Computer won't start"))

help_desk.push(Ticket(3, "Charlie", "Software installation issue"))

help_desk.push(Ticket(4, "David", "Network connectivity problem"))

help_desk.push(Ticket(5, "Eve", "Printer not working"))

print("Current Tickets in Stack:")

for ticket in help_desk.stack:

    print(f"Ticket ID: {ticket.ticket_id}, User: {ticket.user_name}, Issue: {ticket.issue_description}")

print("\nResolving Tickets:")

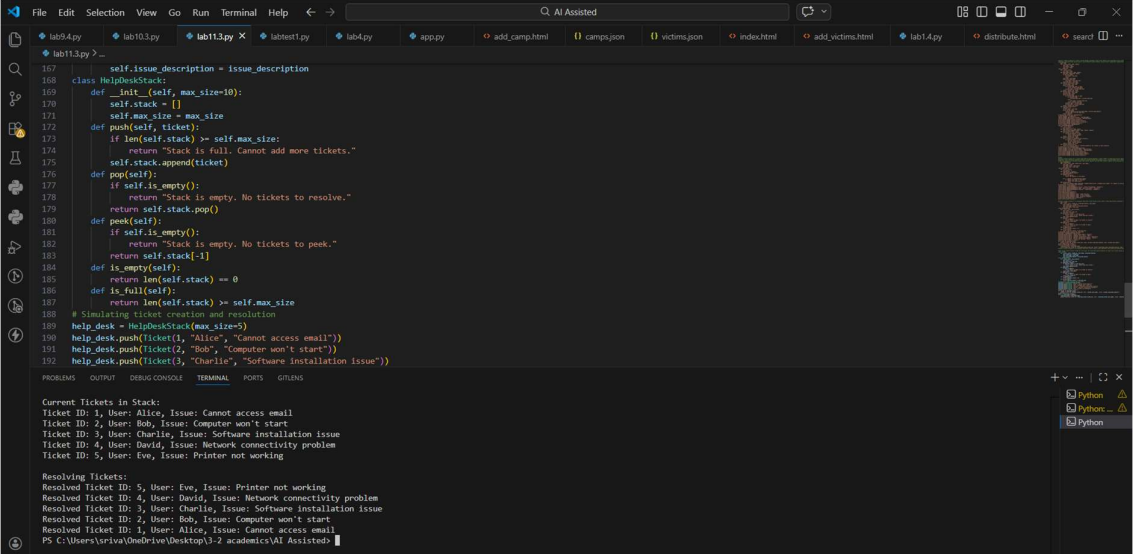
while not help_desk.is_empty():

    resolved_ticket = help_desk.pop()

    print(f"Resolved Ticket ID: {resolved_ticket.ticket_id}, User: {resolved_ticket.user_name},
Issue: {resolved_ticket.issue_description}")

```

OUTPUT:



The screenshot shows a VS Code editor with a Python script named `lab113.py` and its output in the terminal. The script defines a `HelpDeskStack` class with methods for pushing, popping, and checking the stack. It then simulates ticket creation and resolution.

```

157 self.issue_description = issue_description
158 class HelpDeskStack:
159     def __init__(self, max_size=10):
160         self.stack = []
161         self.max_size = max_size
162     def push(self, ticket):
163         if len(self.stack) >= self.max_size:
164             return "Stack is full. Cannot add more tickets."
165         self.stack.append(ticket)
166     def pop(self):
167         if self.is_empty():
168             return "Stack is empty. No tickets to resolve."
169         return self.stack.pop()
170     def peek(self):
171         if self.is_empty():
172             return "Stack is empty. No tickets to peek."
173         return self.stack[-1]
174     def is_empty(self):
175         return len(self.stack) == 0
176     def is_full(self):
177         return len(self.stack) >= self.max_size
178
179 # Simulating ticket creation and resolution
180 help_desk = HelpDeskStack(max_size=5)
181 help_desk.push(Ticket(1, "Alice", "Cannot access email"))
182 help_desk.push(Ticket(2, "Bob", "Computer won't start"))
183 help_desk.push(Ticket(3, "Charlie", "Software installation issue"))

```

The terminal output shows the current tickets in the stack and the resolved tickets:

```

Current Tickets in Stack:
Ticket ID: 1, User: Alice, Issue: Cannot access email
Ticket ID: 2, User: Bob, Issue: Computer won't start
Ticket ID: 3, User: Charlie, Issue: Software installation issue
Ticket ID: 4, User: David, Issue: Network connectivity problem
Ticket ID: 5, User: Eve, Issue: Printer not working

Resolving Tickets:
Resolved Ticket ID: 5, User: Eve, Issue: Printer not working
Resolved Ticket ID: 4, User: David, Issue: Network connectivity problem
Resolved Ticket ID: 3, User: Charlie, Issue: Software installation issue
Resolved Ticket ID: 2, User: Bob, Issue: Computer won't start
Resolved Ticket ID: 1, User: Alice, Issue: Cannot access email

```

Analysis:

- The system uses a stack data structure to store help desk tickets. Tickets are added using `push()` and resolved using `pop()`.
- The ticket system follows Last-In, First-Out (LIFO), meaning the most recently added ticket is resolved first.
- Functions like `peek()`, `is_empty()`, and `is_full()` help check the top ticket and manage stack capacity efficiently.

Task-5

PROMPT:

Create a Campus Resource Management System design. Student Attendance Tracking Event Registration System Library Book Borrowing Bus Scheduling System Cafeteria Order.

CODE:

```
class EventRegistrationQueue:

    def __init__(self):

        self.queue = []

    def enqueue(self, registration):

        self.queue.append(registration)

    def dequeue(self):

        if not self.queue:

            return "No registrations in the queue"

        return self.queue.pop(0)

    def display(self):

        return [f"{registration['name']} registered for {registration['event']}" for registration in
self.queue]

# Testing the Event Registration System

event_queue = EventRegistrationQueue()

event_queue.enqueue({"name": "Alice", "event": "Tech Conference"})

event_queue.enqueue({"name": "Bob", "event": "Art Workshop"})

event_queue.enqueue({"name": "Charlie", "event": "Music Festival"})

print("Current Registrations:")

print(event_queue.display())

print("\nProcessing Registrations:")
```

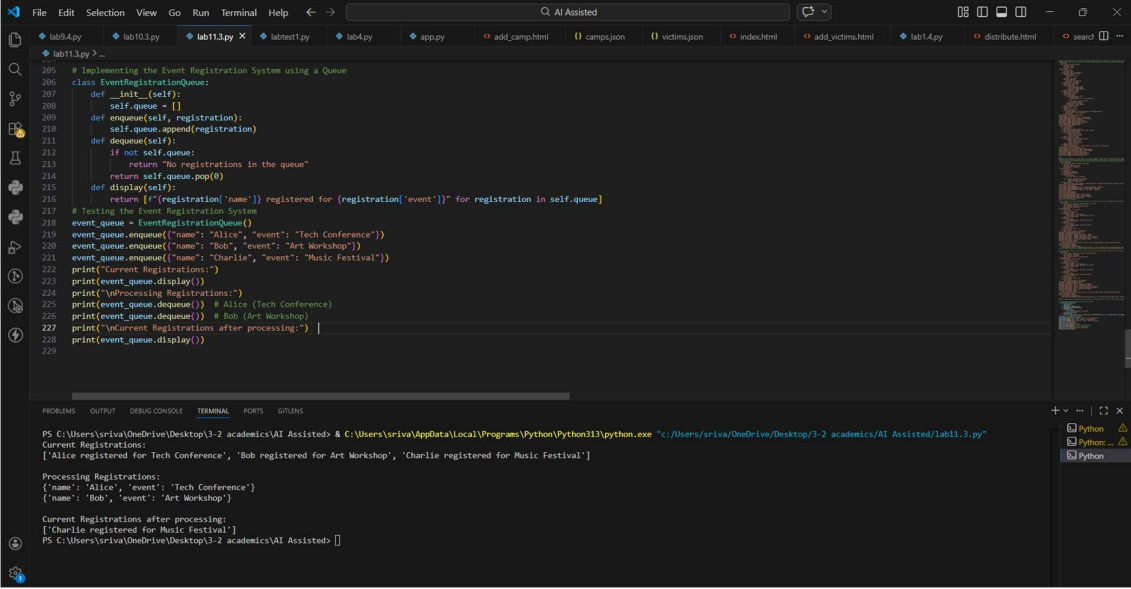
```
print(event_queue.dequeue()) # Alice (Tech Conference)
```

```
print(event_queue.dequeue()) # Bob (Art Workshop)
```

```
print("\nCurrent Registrations after processing:")
```

```
print(event_queue.display())
```

OUTPUT:



The screenshot shows a VS Code editor with a Python file named `lab113.py`. The code implements an event registration system using a queue. It defines an `EventRegistrationQueue` class with methods for enqueueing, dequeueing, and displaying registrations. The main script tests the system by enqueueing three registrations: Alice for a Tech Conference, Bob for an Art Workshop, and Charlie for a Music Festival. It then processes the registrations in order, printing the current state of the queue at each step.

```
205 # Implementing the Event Registration System using a Queue
206 class EventRegistrationQueue:
207     def __init__(self):
208         self.queue = []
209     def enqueue(self, registration):
210         self.queue.append(registration)
211     def dequeue(self):
212         if not self.queue:
213             return "No registrations in the queue"
214         return self.queue.pop(0)
215     def display(self):
216         return [{"registration['name']} registered for {registration['event']} for registration in self.queue]
217
218 # Testing the Event Registration System
219 event_queue = EventRegistrationQueue()
220 event_queue.enqueue({"name": "Alice", "event": "Tech Conference"})
221 event_queue.enqueue({"name": "Bob", "event": "Art Workshop"})
222 event_queue.enqueue({"name": "Charlie", "event": "Music Festival"})
223 print("Current Registrations:")
224 print(event_queue.display())
225 print("\nProcessing Registrations:")
226 print(event_queue.dequeue()) # Alice (Tech Conference)
227 print(event_queue.dequeue()) # Bob (Art Workshop)
228 print("\nCurrent Registrations after processing:")
229 print(event_queue.display())
```

The terminal output shows the following sequence of events:

```
PS C:\Users\sriva\OneDrive\Desktop\3-2 academics\AI Assisted> & C:\Users\sriva\AppData\Local\Programs\Python\Python311\python.exe -c:./Users/sriva/OneDrive/Desktop/3-2 academics/AI Assisted/lab11.3.py
Current Registrations:
[{'name': 'Alice', 'event': 'Tech Conference'}, {'name': 'Bob', 'event': 'Art Workshop'}, {'name': 'Charlie', 'event': 'Music Festival'}]

Processing Registrations:
{'name': 'Alice', 'event': 'Tech Conference'}
{'name': 'Bob', 'event': 'Art Workshop'}

Current Registrations after processing:
[{'name': 'Charlie', 'event': 'Music Festival'}]
PS C:\Users\sriva\OneDrive\Desktop\3-2 academics\AI Assisted>
```

Analysis:

The event registration system uses a Queue (FIFO) where students register using `enqueue()` and registrations are processed using `dequeue()`.

The system follows the First-In, First-Out principle, meaning the first student who registers for an event is processed first, ensuring fair registration handling.